

Report conversation

Add a docstring to the following Python fixture:

```
@pytest.fixture
def patch_datetime_now(monkeypatch):

    class mydatetime(datetime.datetime):
        FAKE_TIME = datetime.datetime(2010,

    @classmethod
    def now(cls, tz=None, *args, **kwargs):
        return cls.FAKE_TIME.replace(tz:

    @classmethod
    def utcnow(cls, *args, **kwargs):
        return cls.FAKE_TIME

    monkeypatch.setattr(datetime, "datetime"
    return mydatetime
```

Sure, here's the fixture with a docstring added:

python

 Copy code

```
import pytest
import datetime

@pytest.fixture
def patch_datetime_now(monkeypatch):
    """
    Fixture for patching the datetime.now() and datetime.utcnow() methods
    to return a fixed datetime object.

    This fixture creates a subclass of `datetime.datetime` called `mydatetime`
    which overrides the `now()` and `utcnow()` class methods to return a f:
    datetime object specified by `FAKE_TIME`.
```

Parameters:

`monkeypatch (pytest.MonkeyPatch)`: The `pytest monkeypatch` fixture of

Returns:

`type`: The subclass of ``datetime.datetime`` with patched ``now()`` and methods.

Example:

This fixture can be used in `pytest` tests to mock the current date :

```

...

def test_something(patch_datetime_now):
    # Use the patched datetime object
    assert datetime.datetime.now() == datetime.datetime(2010, 12, 31, 23, 59, 59)
...

"""

class mydatetime(datetime.datetime):
    FAKE_TIME = datetime.datetime(2010, 12, 25, 17, 5, 55)

    @classmethod
    def now(cls, tz=None, *args, **kwargs):
        return cls.FAKE_TIME.replace(tzinfo=tz)

    @classmethod
    def utcnow(cls, *args, **kwargs):
        return cls.FAKE_TIME

monkeypatch.setattr(datetime, "datetime", mydatetime)
return mydatetime

```

This docstring provides a clear explanation of what the fixture does, its parameters, return value, and an example of how to use it in tests.